Beágyazott rendszerek tipikus működési módjai

Scherer Balázs



Budapesti Műszaki és Gazdaságtudományi Egyetem (BME),
Mesterséges Intelligencia és Rendszertervezés Tanszék (MIT)

© BME-MIT 2026

Tartalom

- Beágyazott szoftverek jellemző működési módjainak összefoglalása
- Bootloader
- Inicializációs mód
- Szervíz, kalibrációs mód
- Hibamód
  - Funkcionális biztonság áttekintés
  - Minimális elvárás a hibamóddal szemben

© BME-MIT 2026

Tartalom

- **Beágyazott szoftverek jellemző működési módjainak összefoglalása**
- Bootloader
- Inicializációs mód
- Szervíz, kalibrációs mód
- Hibamód
  - Funkcionális biztonság áttekintés
  - Minimális elvárás a hibamóddal szemben

© BME-MIT 2026

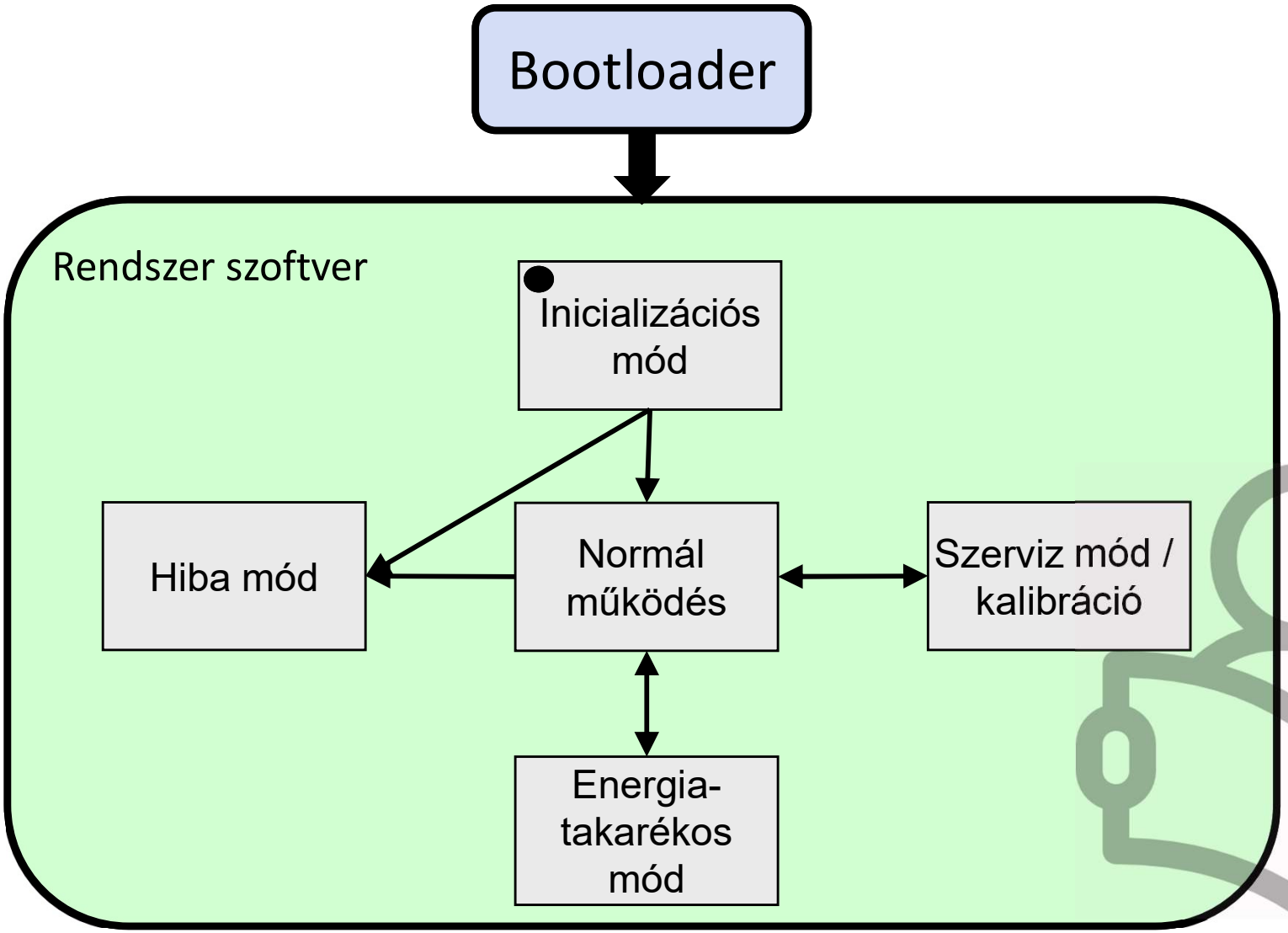
Beágyazott szoftver működési módjai

- Szemben a legtöbb Deszktop-PC-s alkalmazással a beágyazott szoftvert rendszerként kell kezelni, és ez azzal jár, hogy több működési módja is lehet a szituációtól függően
- A működési módok jó része alkalmazás független, bármely beágyazott rendszernél elő fordulhat, de természetesen sok működési módnak a szerepe bizonyos alkalmazásban minimálissá csökken, míg másoké felértékelődik

© BME-MIT 2026

Általános működési módok

- A pontos állapotok és átmenetek rendszer és alkalmazás függők



A működési módok szerepe röviden összefoglalva

- **Bootloader:** külön alkalmazás, feladata a főprogram helyességének ellenőrzése, némely esetben frissítése, és elindítása
- **Inicializációs mód:** Feladata reszet vektor utáni ellenőrzések lefuttatása, a perifériák, lábak alapállapotba helyezése
- **Normál működési mód:** Normál vezérlési, mérési funkcionalítás
- **Diagnosztikai/kalibrációs mód:** Diagnosztikai felület nyújtása, belső állapotváltozók kívülre való elérhetőségének biztosítása, tesztelési beavatkozás biztosítása, kalibrációs eljárások végrehajtása
- **Energia takarékos mód:** A mikrovezérlő valamilyen energiacsökkentett módjába való belépés, a külső áramkörök energia takarékos mód szerinti vezérlése, az ébresztés menedzselése
- **Hiba mód:** A hiba esetén biztonságos állapotba lépés, a hiba okának letárolása

© BME-MIT 2026

Tartalom

- Beágyazott szoftverek jellemző működési módjainak összefoglalása
- **Bootloader**
- Inicializációs mód
- Szervíz, kalibrációs mód
- Hibamód
  - Funkcionális biztonság áttekintés
  - Minimális elvárás a hibamóddal szemben

© BME-MIT 2026

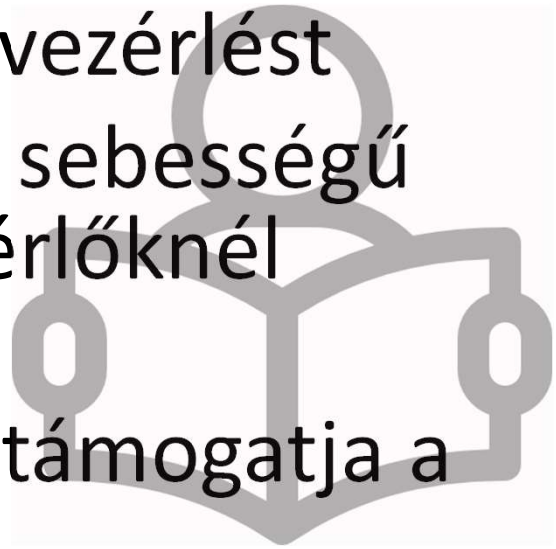
Bootloader szerepe

- A legtöbb szoftverhez előbb utóbb készül új verzió, javítás, ezt valahogy el kell juttatni az eszközhöz
- Nem megengedett sok esetben a doboz megbontása
  - Nem hozzáférhető helyen van
  - Macerás, szakképzett személy kell hozzá
  - Védekezünk a doboz bontás ellen: rosszindulatú támadás lehet
  - A debug interfész a legtöbb termékben le van tiltva (veszélyt jelent: kód kiolvasás, felülírás)
- A rendszer normál kommunikációs interfészét kell használni a program frissítéséhez



Jellemző Bootloader típusok: **Alkalmazás processzorok bootloader-je**

- Az alkalmazói program nemfelejtő memóriában van tárolva, de onnan direktben nem futtatható (SPI Flash, Lassú külső flash)
- A bootloader feladata, hogy ezt átmásolja a futtatható tartományba (Jellemzően valamilyen RAM-ba), és átadja a vezérlést
- Alkalmazás processzorokra, nagy sebességű rendszerekre jellemző, mikrovezérlőknél nagyon ritka
- Ez a bootloader jellemzően nem támogatja a nemfelejtő memória frissítését



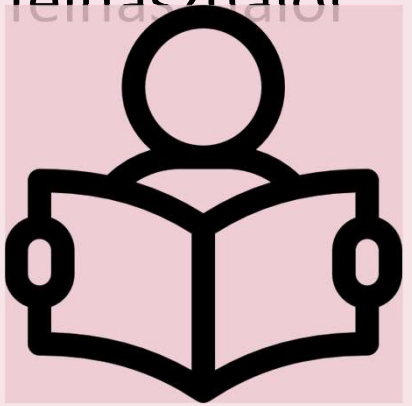
Jellemző Bootloader típusok: **Mikrovezérlős bootloader program frissítés támogatással**

- Reset után jellemzően rögtön ez indul el.
- Valamilyen jelzés hatására elindítja a flash átprogramozását, vagy csak átadja a vezérlést a főprogramnak
  - Külső láb állapot reset alatt
  - A mikrovezérlő nem felejtő memóriájában (általában az ún. Option byte) reszetnél tapasztalt bejegyzés (az alkalmazási program szokta így felkérni a bootloadert szoftver frissítésre).
- A programozáshoz valamilyen normál kommunikációs interfészt használ: UART, CAN, USB
- Az átprogramozás jellemzően nagyon egyszerű protokollon alapul
- Szinte az összes modern mikrovezérlő rendelkezik egy ilyen jellegű bootloader-el, ami reset alatt speciális lábakkal aktiválható

© BME-MIT 2026

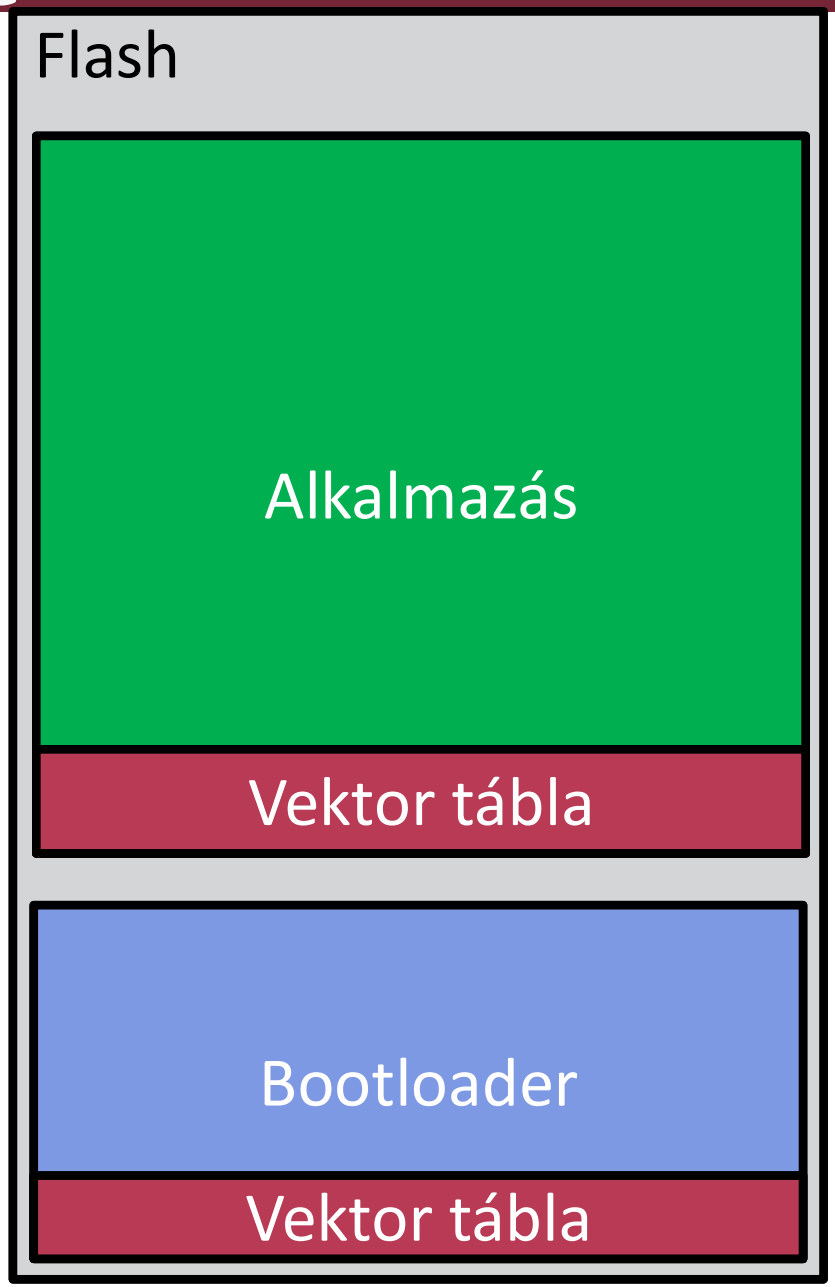
Jellemző Bootloader típusok: **Mikrovezérlős termékbe használt bootloader**

- Reszet után mindig ez indul el.
- Jellemzően nem végez átprogramozást, csak ellenőrzi, hogy a program memória értéke valid-e
  - Checksum, CRC ellenőrzés
  - Hiba esetén hiba mód, biztonságos állapot garantálása, ami lehet az, hogy kétbankos bootloader esetén a régebbi alkalmazás verziót indítjuk el
- Az átprogramozás jellemzően egy diagnosztikai protokoll támogatja az alkalmazási szoftverben
  - Sokkal komplexebb kommunikáció, ami tartalmaz felhasználói azonosítást, jogosultság ellenőrzést
- Ez a jellemző ipari bootloader megvalósítás



Mikrovezérlők általános 1 bankos bootloaderjének felépítése

- **A bootloader indul először**
- Ellenőrzi, hogy van-e újraprogramozás kérés, ha egyszerű program frissítő bootloader-ről van szó és ha kell akkor betölti az új alkalmazást
- Ellenőrzi, hogy van-e valid alkalmazói program és elindítja azt:
- **Az alkalmazás linker szcriptje el van tolva**
- **A bootloader átadja a vezérlést az alkalmazásnak**
  - **A kritikus pont a vektor tábla kicserélése**



A mikrovezérlők általános 2 bankos bootloaderjének felépítése

- A működése megegyezik az előző esettel azzal a különbséggel, hogy el kell döntenie, hogy melyik bank-ban lévő alkalmazást indítja el
  - Ez lehet jelezni a flash egy speciális területén, vagy az alkalmazás egy adott memória címén
- Célja, hogy megakadályozza, hogy a vezérlő valid alkalmazás nélkül maradjon
  - Pl. hiba történik a program frissítés során
- Jellemző működés az ipari eszközöknél, ahol távolról diagnosztikai protokollal frissítjük a programot.



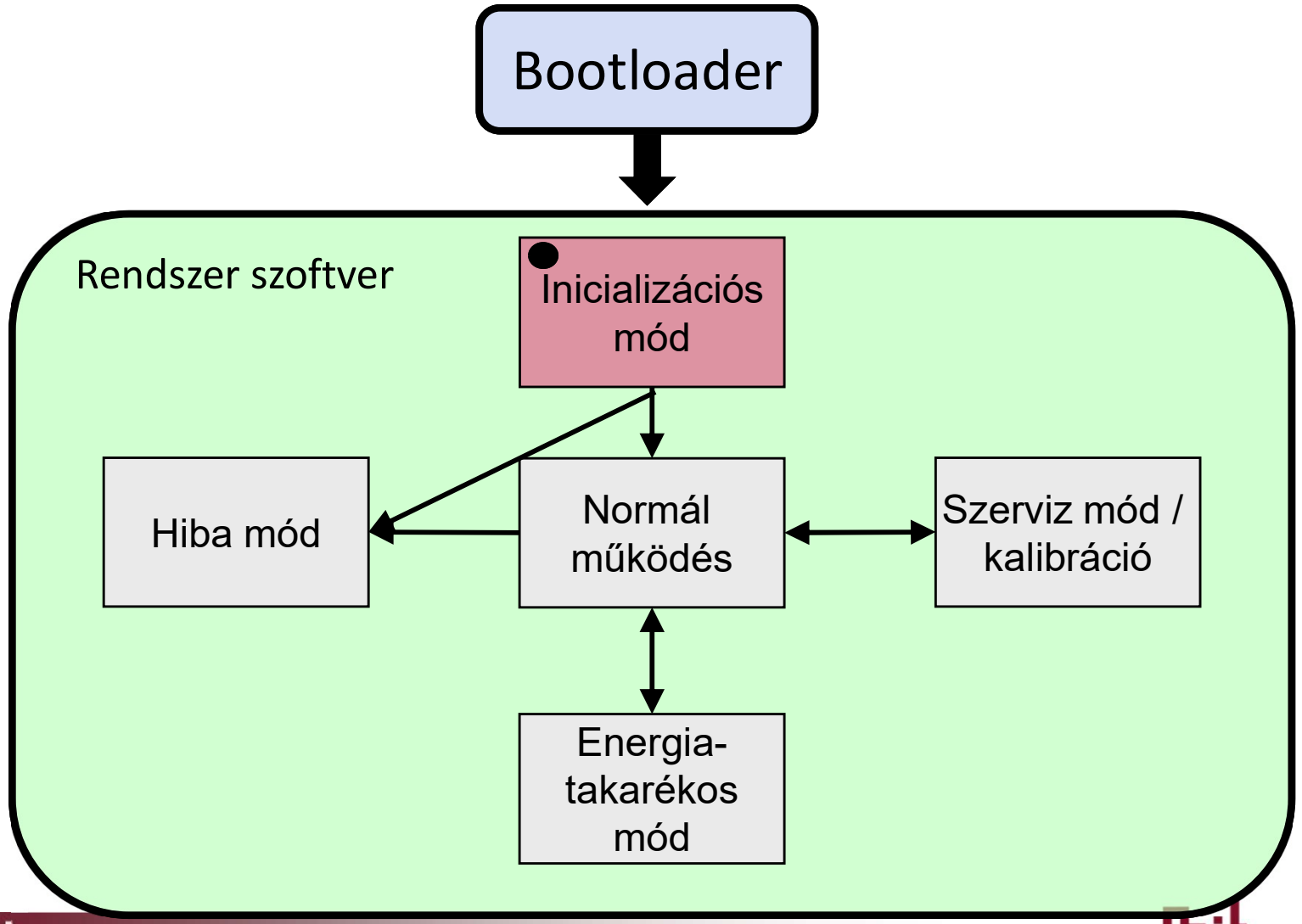
Tartalom

- Beágyazott szoftverek jellemző működési módjainak összefoglalása
- Bootloader
- **Inicializációs mód**
- Szervíz, kalibrációs mód
- Hibamód
  - Funkcionális biztonság áttekintés
  - Minimális elvárás a hibamóddal szemben

© BME-MIT 2026

Inivializációs mód

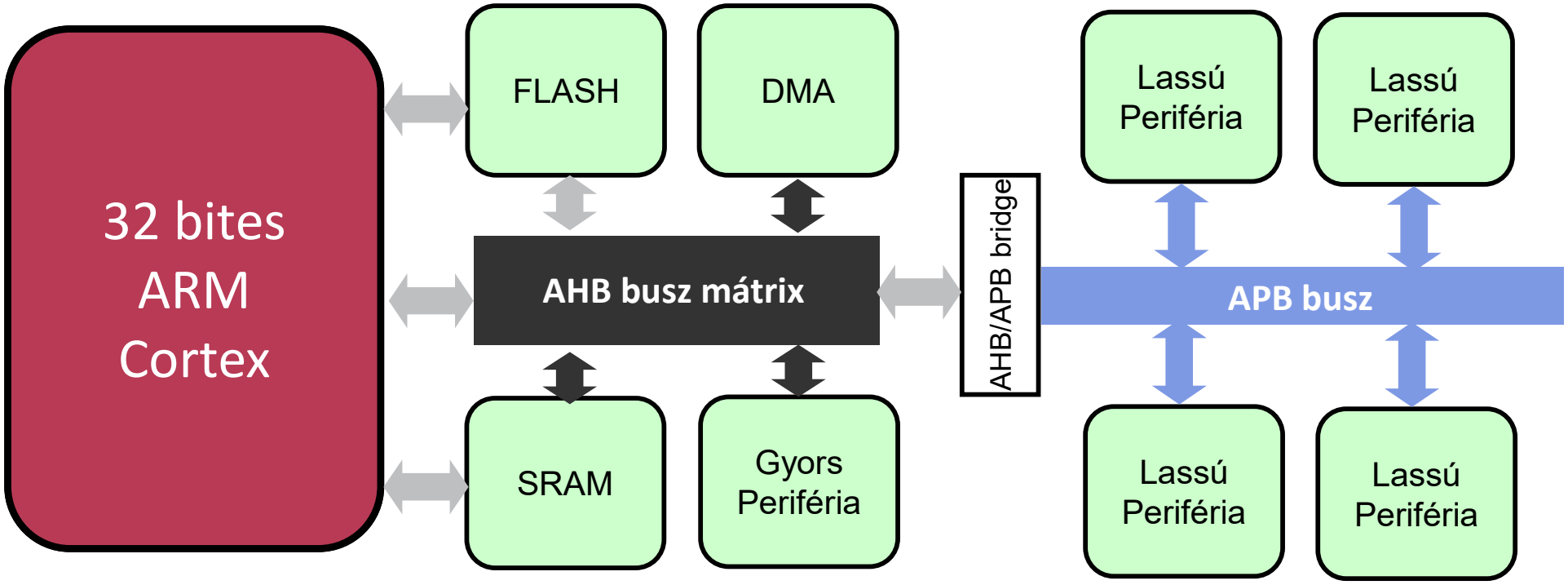
- Az inicializációs mód nagyon függ az alkalmazási kőrtől és az alkalmazott vezérlő/hardware típusától



Tipikus lépések inicializációs módban 1.

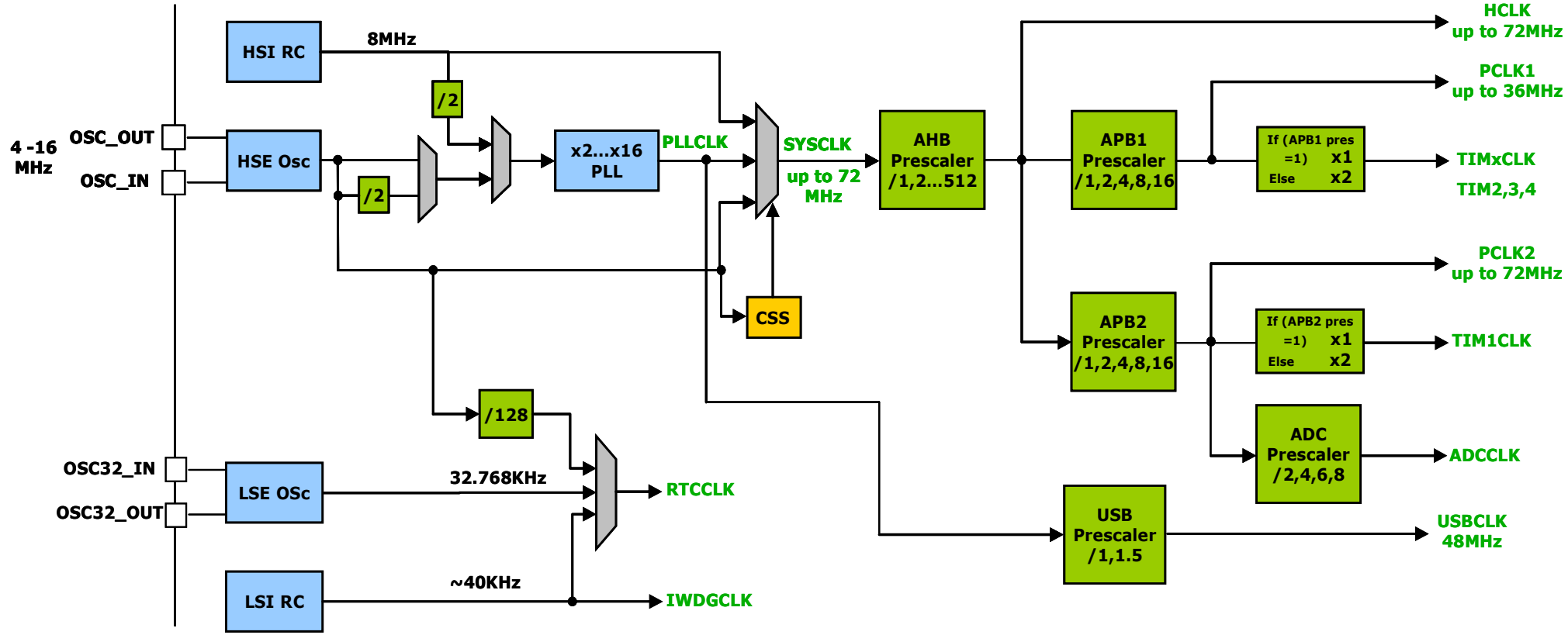
- **Vezérlő szintű inicializációk, öntesztek**
  - **Vezérlő önteszt** (safety critical rendszerekben)**:** ALU működés egyszerű tesztelése, különböző műveletek halmazok teszt jellegű végrehajtása
  - **Órajel hálózat bekonfigurálása:** órajel források kiválasztása, rendszer órajel források meghatározása, órajel osztó hálózat bekonfigurálása
  - **SRAM tesztelés** (safety critical rendszerekben)**:** széleskörű irodalma van annak, hogy milyen Read-Write patterneket kell végrehajtani dinamikus és statikus memóriák esetében annak kizárására, hogy az eszköz RAM memóriája sérült legyen
  - **Periféria konfigurációk:** órajelek biztosítása, periféria alap paraméterek beállítása

32-bites vezérlők belső felépítése



- Megjelenik két különböző képességű busz AHB - APB
- Egyszerűbb esetben (M0, M0+) az AHB mátrix helyett csak egy busz van

Egyszerűsített tipikus órajel hálózat



Tipikus lépések inicializációs módban 2.

- **Board szintű inicializációk, öntesztek**
  - **Külső tápfeszültségek bekapcsolása és visszamérése:** Jellemzően több szintű tápfeszültség van a kártyán. Pl. 12V morot, vagy szelep vezérléshez, +-5V/12V analóg mérésekhez stb.
  - **Külső chippek inicializálása:** Státusz ellenőrzés.
  - **Szenzor inicializáció:** Hihetőség vizsgálat (Pl. hőmérő szenzor induláskor 120C fölötti értéked ad akkor kezdjünk el gyanakodni).
  - **Beavatkozó inicializáció:** Beavatkozók inicializálása, státusz ellenőrzése, és hihetőség vizsgálata (akár rövid idejű bekapcsolással is).
  - **Kommunikációs interfész tesz:** Egyszerű hallgatódzás, vagy próba üzenet, vagy akár ha van akkor loopback teszt

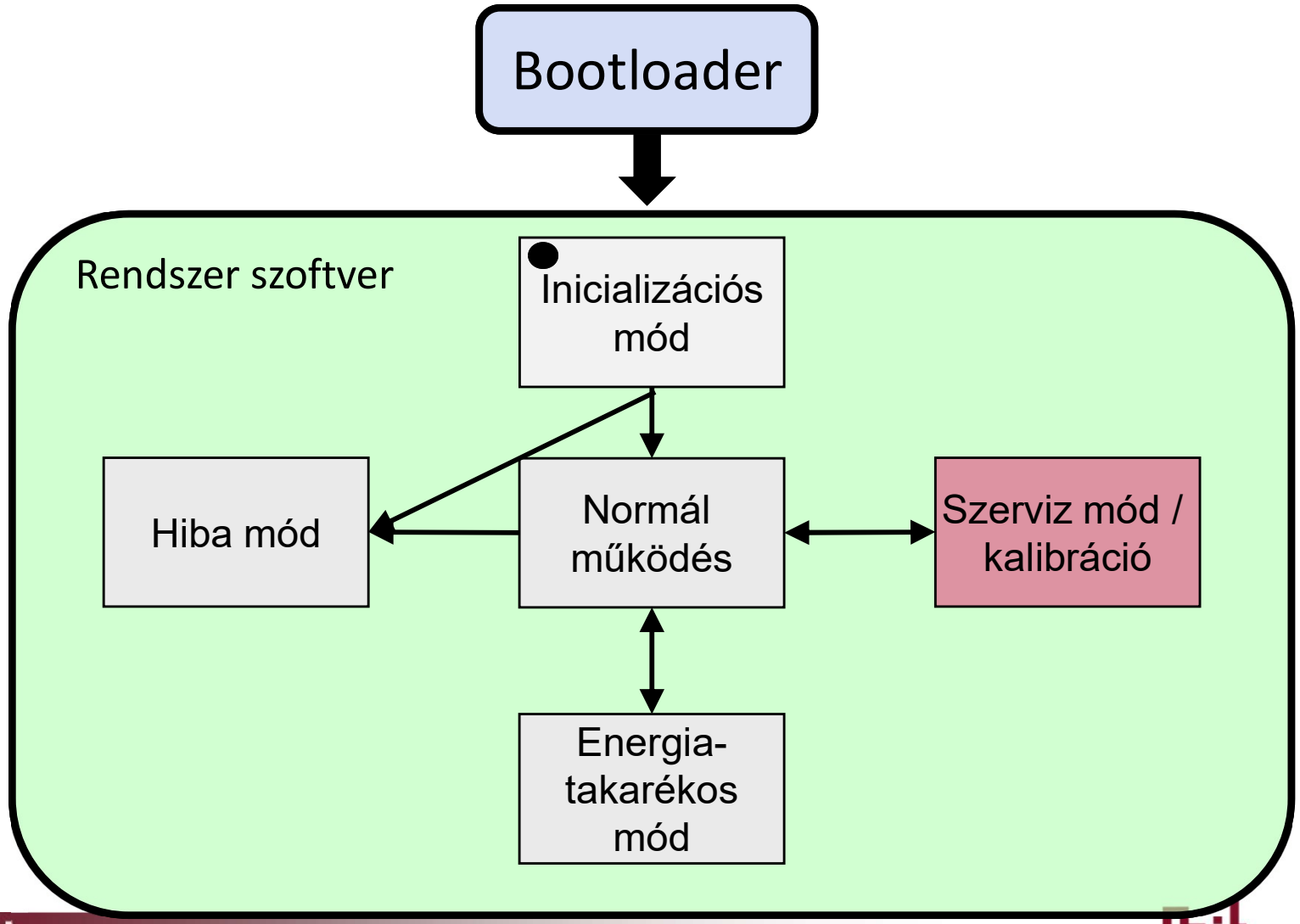
© BME-MIT 2026

Tartalom

- Beágyazott szoftverek jellemző működési módjainak összefoglalása
- Bootloader
- Inicializációs mód
- **Szervíz, kalibrációs mód**
- Hibamód
  - Funkcionális biztonság áttekintés
  - Minimális elvárás a hibamóddal szemben

Szerviz mód / kalibráció

- A legtöbb ipari egységben van valami kiterjesztett ellenőrzést lehetővé tevő mód



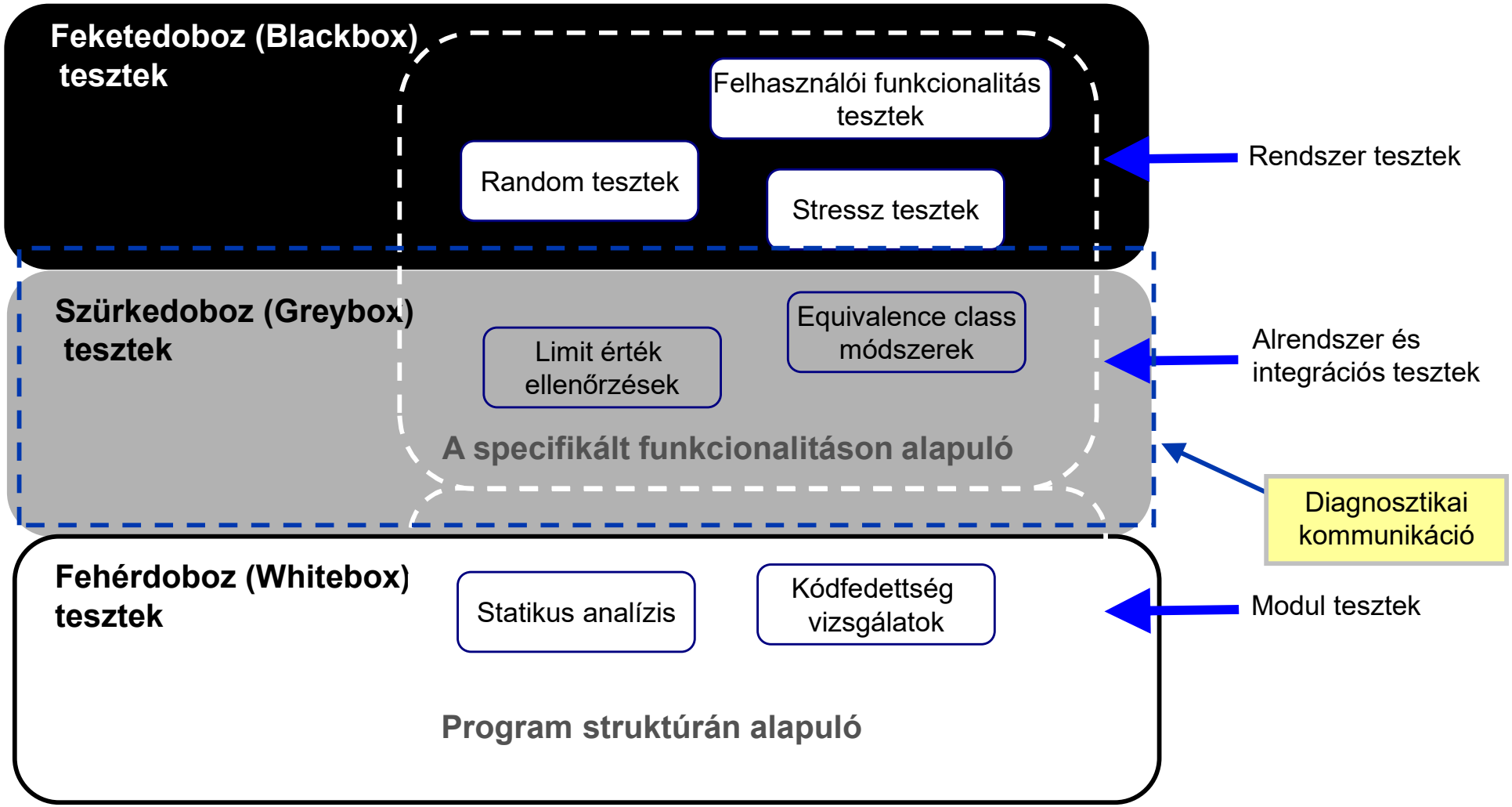
Tipikus funkciók szerviz/kalibrációs módban (Néhány alkalmazásban ez a két mód elkülönül)

- Diagnosztikai kommunikáció biztosítása
  - Felhasználói jogok, azonosítás, belépés biztosítása
  - Hibatároló kiolvasása
  - Státusz lekérdezések
  - Speciális parancsok adása
  - Szoftver frissítés biztosítása (nem feltétlenül szükséges kiterjedt diagnosztikai kommunikáció hozzá, lehet, hogy csak ez az egy funkció van megvalósítva)
- Szenzorok és beavatkozók kalibrálásának támogatása
  - Kalibrációs végrehajtandó szekvencia
  - És/vagy kalibrációs adatok átadása a vezérlőnek (ez nem az adott mérési elrendezésre, hanem a konkrét szenzorra vonatkozik)

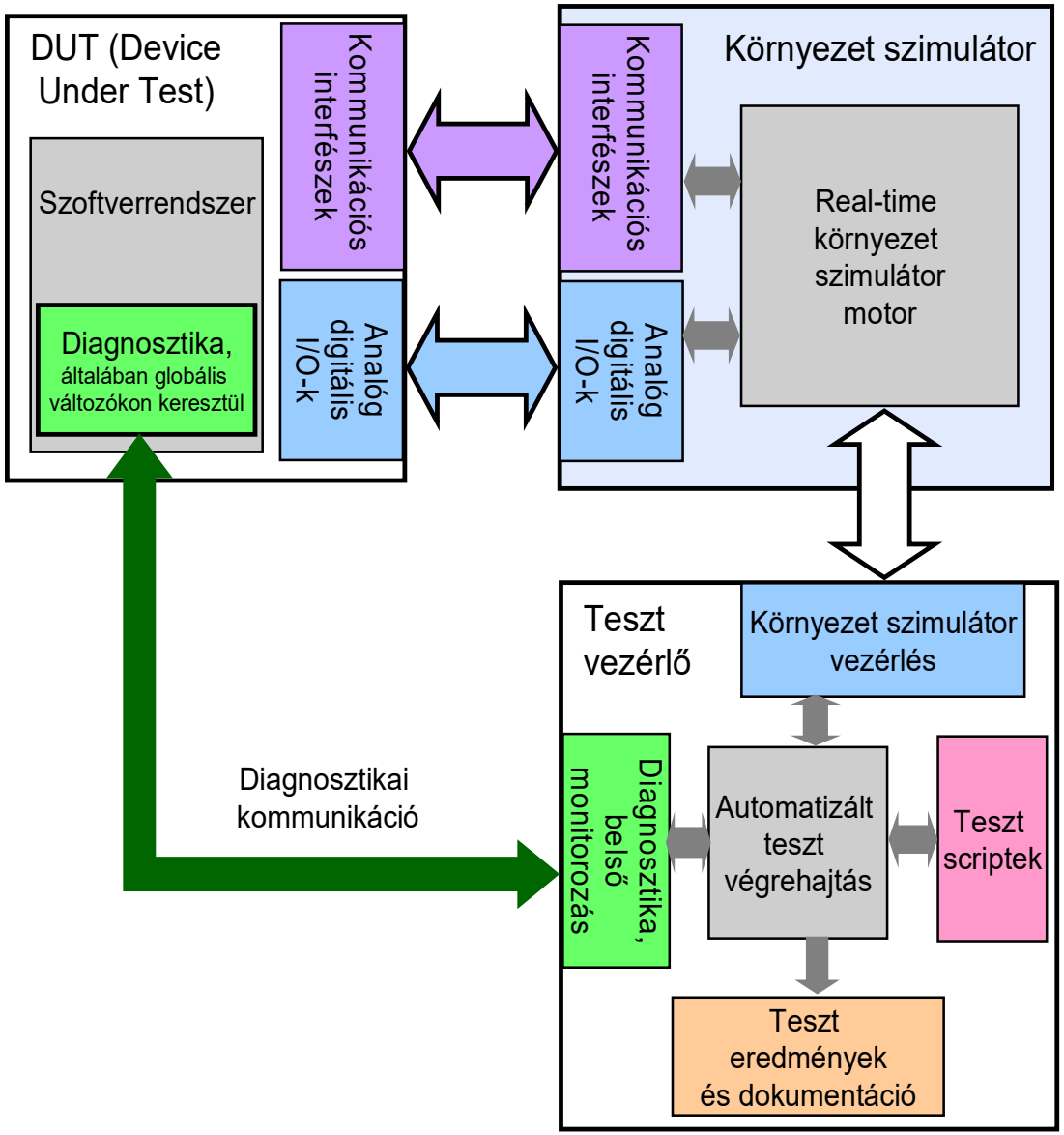


Példa a diagnosztikai kommunikáció szerepére autóipari fejlesztésnél, tesztelésnél

Tesztelés


Feketedoboz (Blackbox) tesztek
Felhasználói funkcionalitás tesztek
Random tesztek
Stressz tesztek
Rendszer tesztek
Szürkedoboz (Greybox) tesztek
Limit érték ellenőrzések
Equivalence class módszerek
A specifikált funkcionalitáson alapuló
Alrendszer és integrációs tesztek
Diagnosztikai kommunikáció
Fehérdoboz (Whitebox) tesztek
Statikus analízis
Kódfedettség vizsgálatok
Modul tesztek
Program struktúrán alapuló

Tipikus fejlesztési tesztkörnyezet szürke és feketedoboz tesztekhez



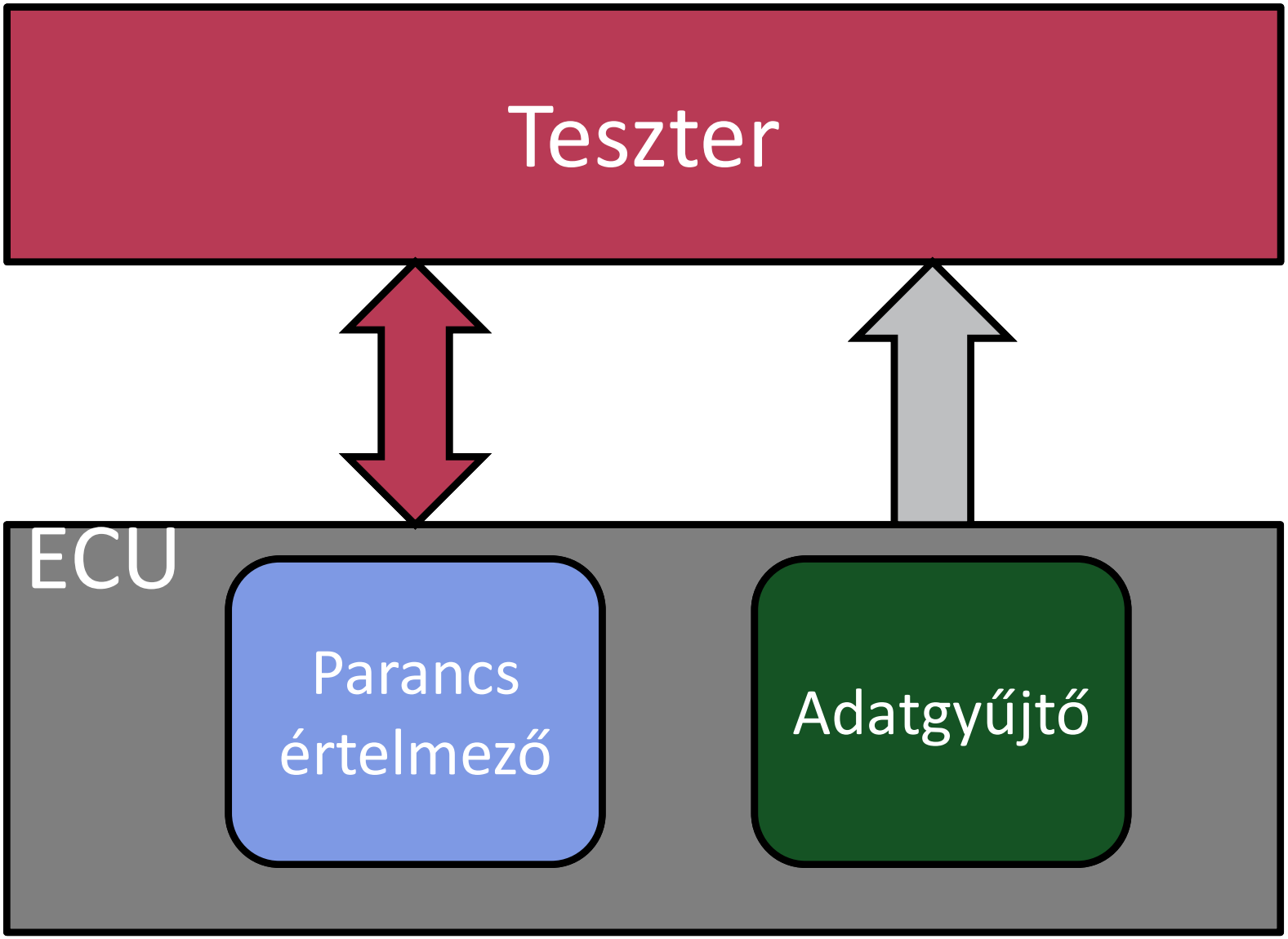
A diagnosztikai kommunikációk célja

- ECU megfigyelése:
  - Folyamatos adatgyűjtés
  - Polling alapú adatgyűjtés
- ECU vezérlése
  - Adatok, konstansok, kalibrációs adatok megváltoztatása
- ECU programmemória frissítése
- Felhasználó azonosítása

- Példa diagnosztikai / kalibrációs protokollokra:
  - UDS (Unified Diagnostic Services)
  - XCP (Universal Measurement and Calibration Protocol)

© BME-MIT 2026

A diagnosztikai kommunikáció jellemző blokkjai



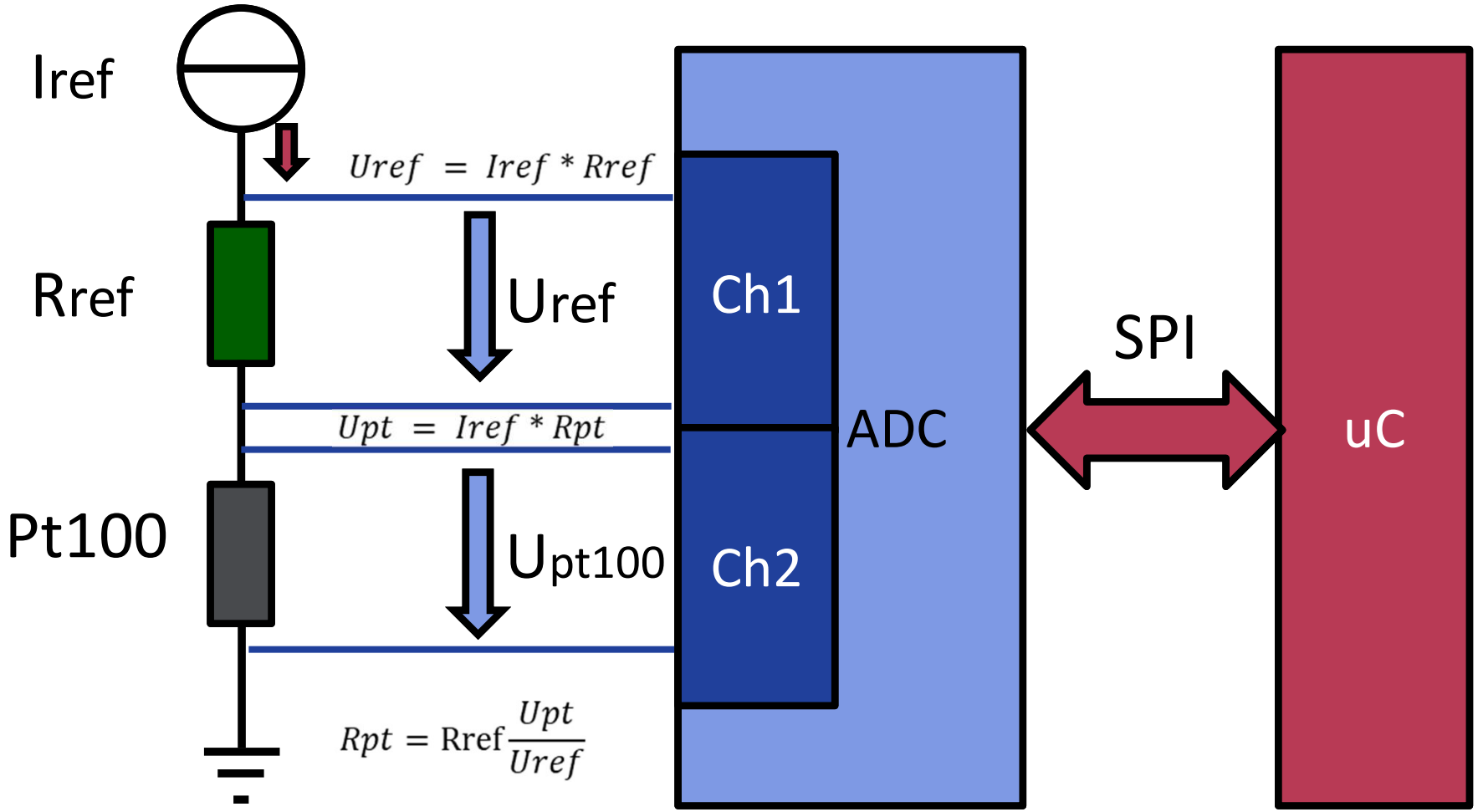
Példa a kallibráció fontosságára

Kalibráció szükségessége a mérés során

- Az egyes mérési átalakítók paraméterei elmászhatnak
  - hőmérséklet és idő függők
  - Offszet, drift, egyéb hibák keletkezhetnek
- Ha lehet minél egyszerűbb referenciára vezessük vissza a mérésünket, úgy, hogy a mérés közben az átalakítók hibáit ejtsük ki
- Idő, vagy ellenállás referencia a lehető legegyszerűbb
  - De ezeknek is van induló pontatlansága és arrébb is mászhatnak

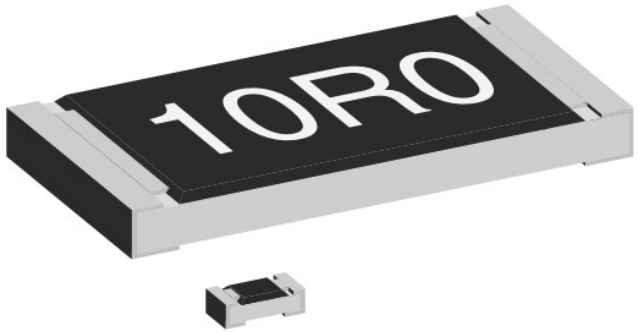
© BME-MIT 2026

Példa: Hőmérsékletet szeretnénk mérni Pt100 ellenállással, javasolt elrendezés

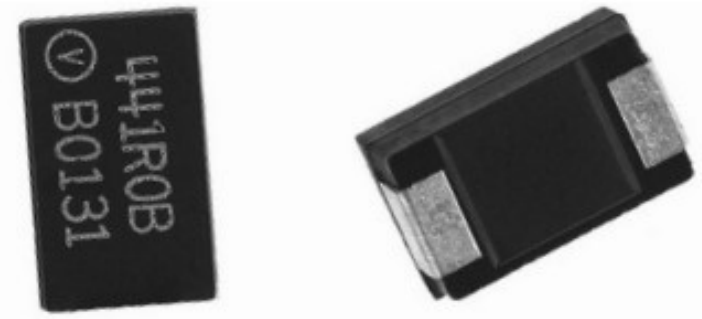


Példa: a referencia ellenállás pontossága

- 1%-os referencia
  - Pár forint
  - Induló pontatlanság
  - Nagy hőfüggés: +/- 100ppm/K
  - Nagy öregedés: $\Delta R/R$ = 1 % for 1000 h at 70 ° C



- 0,01%-os igazi referencia
  - 10e+ Ft
  - Nagy induló pontosság
  - Kis hőfüggés: +/- 2ppm/K
  - Kis öregedés: ± 0.005 % 2000h 70C



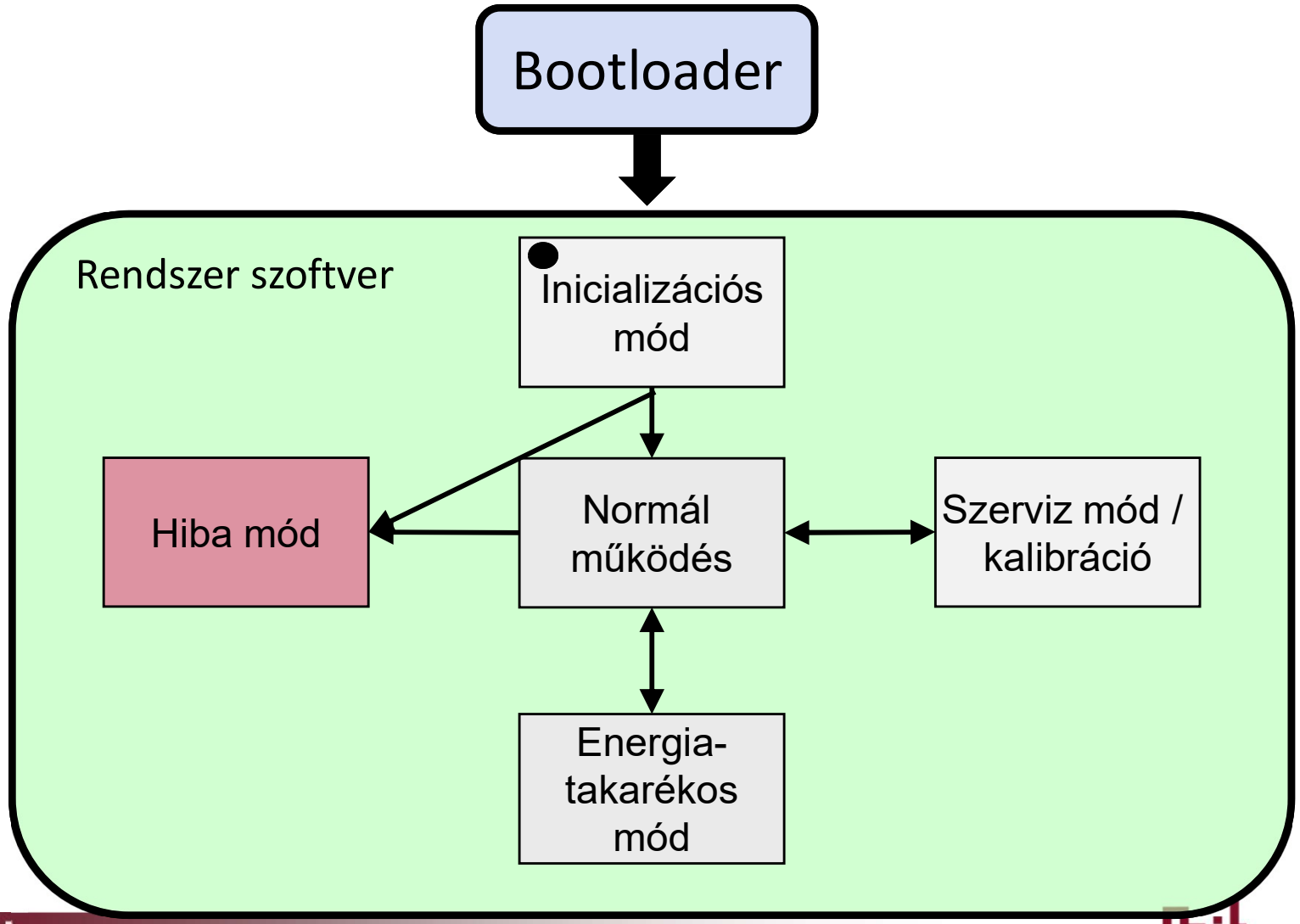
Tartalom

- Beágyazott szoftverek jellemző működési módjainak összefoglalása
- Bootloader
- Inicializációs mód
- Szervíz, kalibrációs mód
- **Hibamód**
  - Funkcionális biztonság áttekintés
  - Minimális elvárás a hibamóddal szemben

© BME-MIT 2026

Hiba mód

- Minden beágyazott rendszer számára alapkövetelmény, és elengedhetetlen hiba esetén a biztonságos viselkedés



Tartalom

- Beágyazott szoftverek jellemző működési módjainak összefoglalása
- Bootloader
- Inicializációs mód
- Szervíz, kalibrációs mód
- **Hibamód**
  - **Funkcionális biztonság áttekintés**
  - Minimális elvárás a hibamóddal szemben

© BME-MIT 2026

Funkcionális biztonság

- A biztonságosság a megbízhatósággal és a rendelkezésre állással szemben nem azzal foglalkozik, hogy a rendszer működik-e, vagy nem, hanem azzal, hogy okoz-e valami kárt vagy veszélyt, hogyha a rendszer nem vagy nem jól működik.

- Ha a rendszer elromlik az csak veszélyt jelent, nem feltétlenül kárt. A biztonságossággal foglalkozó részek feladata az, hogy megtalálja az elviselhető kockázati határt (limit risk).

© BME-MIT 2026

A hibák hármas modellezési szintje

- **Fault – hibaok**
  - Az az elsődleges (bizonyított vagy feltételezett ok), ami error-hoz vezet.
- **Error – hiba**
  - Az a rendszerállapot, ami hibajelenséghez vezethet.
- **Failure – hibajelenség**
  - A szolgáltatásokban megjelenő hiba.

© BME-MIT 2026

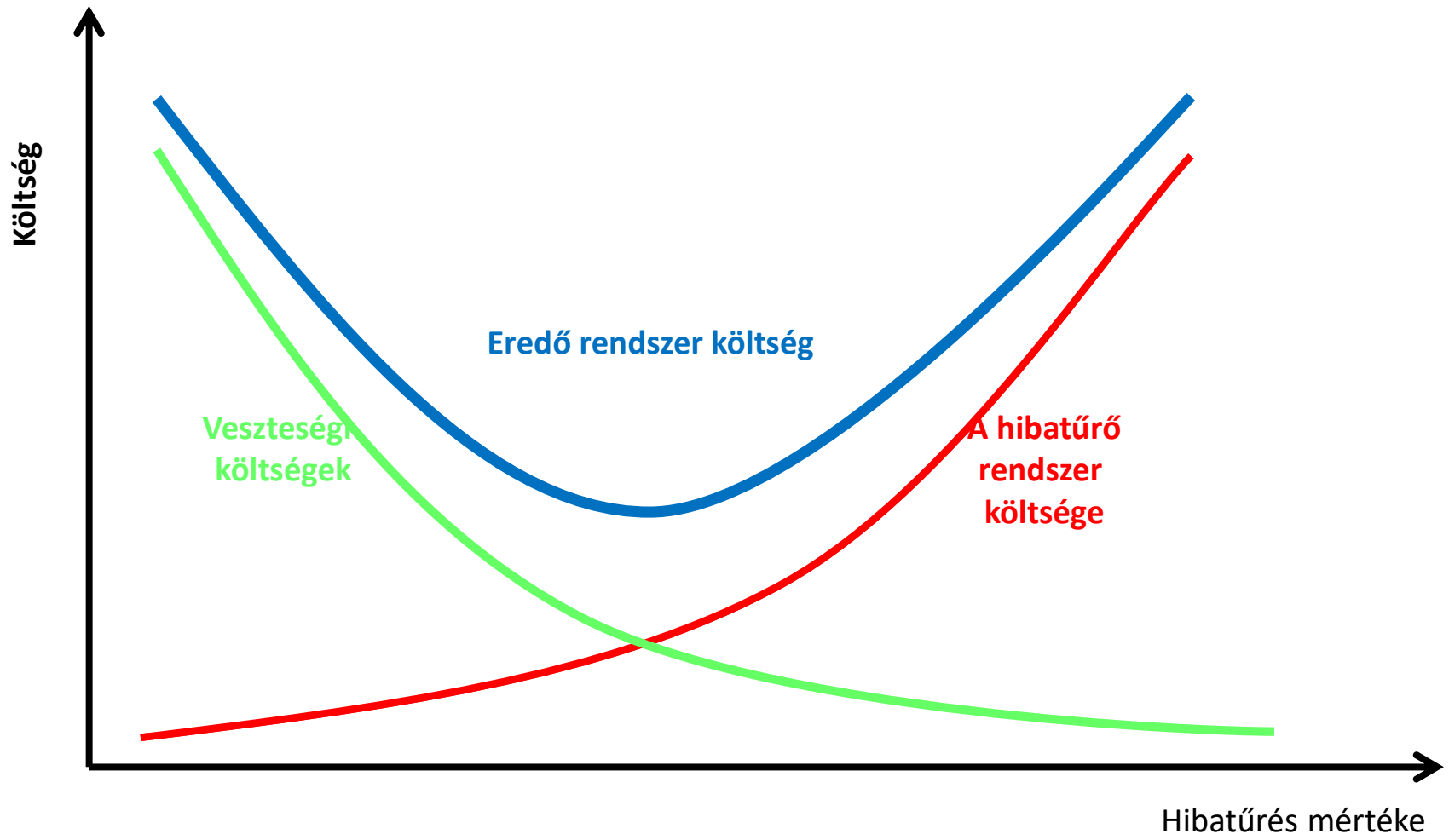
Cél: a hibajelenség (failure) elkerülése

- Konstruktív módszerek
  - Hibamegelőzés (fault prevention)
    - Nyers erő módszer, pl. árnyékolás, robusztus alkatrész, jó tápegység.
  - Hibatűrés (fault tolerance)
    - Annak megakadályozása, hogy a bekövetkezett hibaok (fault) által létrehozott hiba (error) hibajelenséghez (failure-höz) vezessen.
- Analitikus módszerek
  - Hibamegszüntetés (fault removal)
    - A hiba hatását csökkenti a bekövetkezés utáni minél korábbi megszüntetés, diagnosztika, önellenőrzés --> javítás.
  - Hiba-előrejelzés (fault forecasting)
    - Nyilván jó, ha a rendszer hibáit előre lehet jelezni, ez öregedő típusú alkatrészeknél hatékony módszer.

© BME-MIT 2026

Kockázat, és kockázati optimum

$R_{kockázat} = P_{\text{hiba bekövetkezésének valószínűsége}} * L_{\text{a hiba által okozott veszteség}}$



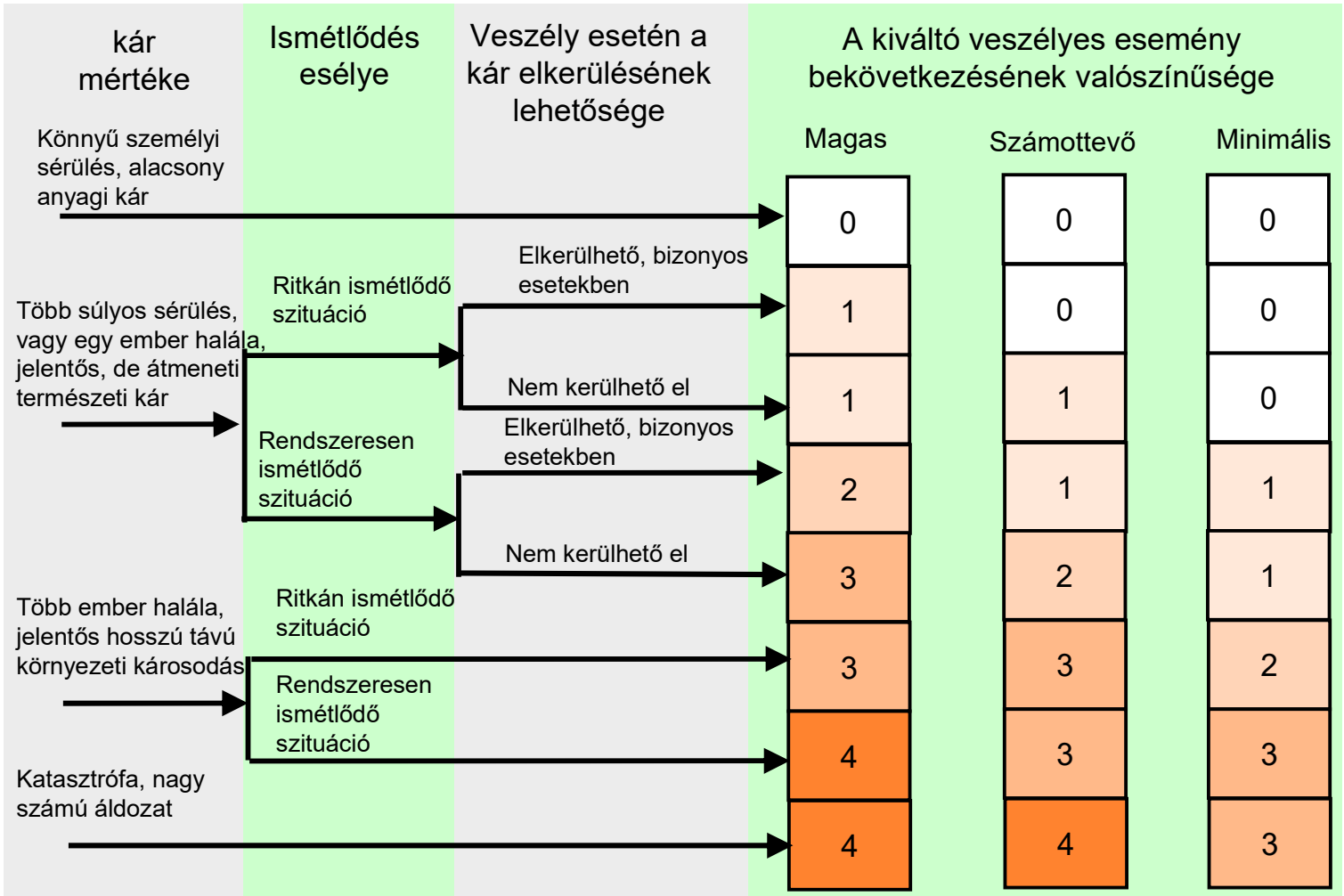
Funkcionális biztonság

- A biztonságosság a megbízhatósággal és a rendelkezésre állással szemben nem azzal foglalkozik, hogy a rendszer működik-e, vagy nem, hanem azzal, hogy okoz-e valami kárt vagy veszélyt, hogyha a rendszer nem vagy nem jól működik.
- Ha a rendszer elromlik az csak veszélyt jelent, nem feltétlenül kárt. **A biztonságossággal foglalkozó részek feladata az, hogy megtalálja az elviselhető kockázati határt (limit risk).**
- **Valahogy osztályoznunk kell a kockázatot és meg kell határoznunk az adott kockázathoz szükséges minimális védelmet: Safety Integrity Level**



Safety Integrity level meghatározása

- SIL Safety Integrity Level: IEC 61508 példa a meghatározásra
- **A SIL réteg meghatározza a később végrehajtandó fejlesztési és tesztelési eljárásokat**



SIL szintek és megbízhatóság

- Megbízhatóság önmagában nem elég
  - Safe Failure Fraction (SFF): a biztonságos és nem biztonságos hibák aránya
  - Az egész fejlesztési folyamat megbízhatósága, verifikálása, validálása is lényeges.

SIL	Low demand mode: average probability of failure on demand	High demand or continuous mode: probability of dangerous failure per hour
1	$\geq 10^{-2}$ to $< 10^{-1}$	$\geq 10^{-6}$ to $< 10^{-5}$
2	$\geq 10^{-3}$ to $< 10^{-2}$	$\geq 10^{-7}$ to $< 10^{-6}$
3	$\geq 10^{-4}$ to $< 10^{-3}$	$\geq 10^{-8}$ to $< 10^{-7}$
4	$\geq 10^{-5}$ to $< 10^{-4}$	$\geq 10^{-9}$ to $< 10^{-8}$

Hibatűrés

A hiba felfedezése (fault recognition, detection)

- A legelterjedtebb hibadiagnosztikai eljárások:
  - Reference value check
    - Egy ismert gerjesztésre a rendszernek egy ismert választ kell adnia, ha ez nem teljesül, akkor a rendszer rossz.
  - Rendundant value check
    - Egy folyamat eredménye két helyen áll elő, és ezt a két eredményt hasonlítjuk össze. (Például ugyanazt a számítást két különböző algoritmuson alapuló függvény hajtja végre, vagy akár ugyanazt az folyamatot két külön mikrovezérlő hajtja végre.)
  - Kommunikáció ellenőrzése
    - CRC kódok, hibaszámlálók, ACK stb.
  - Fizikai tulajdonságok monitorozása
    - Szenzorok, motorok hőmegfutásának vizsgálata, tápfeszültségük figyelése stb.
  - Programvégrehajtás monitorozása
    - Watchdog timer, watchdog processor



Hiba filterezés

- A legtöbb esetben nem rögtön ütjük be a hibát a hibatárolóba (pl. kommunikációs csomag sérülés)
  - Tranziens hibák miatt hamar téves riasztás történhetne

- A legtöbb esetben minden tipikus hibához egy hibaszámlálót társítanak
  - Ha hiba történik növelik
  - Sikeres végrehajtásnál csökkentik
  - A növelés általában erősebb, mint a csökkentés
  - Ha a számláló elér egy limitet akkor warning, ha még tovább lép, akkor lesz belőle igazi hiba



Hiba tároló

- Minden beágyazott rendszer elvárt része
- Nem felejtő belső vagy külső memóriába hiba kódok időponttal, esetleg más részlettel felvezetve
- Diagnosztikai protokollon keresztül kiolvasható legyen



© BME-MIT 2026

Példa hiba fedésre program végrehajtás során

- A hiba fedés mértéke azt mondja meg, hogy az eredő hiba valószínűség hányad részét detektáljuk, és így mennyire fedjük el, a hibát működés során: pl. alacsony 50%, közepes: 90%, magas 99%
- A SIL rétegnél a maradó, el nem fedett hiba számokat kell figyelembe venni, hiszen ezek a veszélyesek

- Példák javasolt diagnosztikai módszerre egy szoftver végrehajtás során
  - ***Normál Watchdog saját órajelforrásról***: Max. alacsony fedettség érhető el vele.
  - ***Ablakozott watchdog saját órajel forrásról:*** Max. közepes fedettség érhető el vele. Nem csak arra képes, hogy egy maximum időt adjon, hanem arra is, hogy ellenőrizze, hogy a törlés egy adott ablakon belül történik-e meg.
  - ***A program végrehajtás logikai monitorozása:*** Max. közepes fedettség érhető el vele. Annak loggolása, számlálása, hogy az adott függvényeket végrehajtották-e és ha igen hányszor.
  - ***A program végrehajtás időbeli és logikai monitorozása:*** Magas fedettség érhető el vele. Watchdog + logikai monitorozás.

A hibatűrés alapja: redundancia (tartalék)

- Tér-redundancia
  - Információ-redundancia
    (pl. hibavédő kódok, redundáns adatbázisok)
  - Hardver-redundancia
    (pl. többszörözés, belső ellenőrző áramkörök)
  - Szoftver-redundancia
    (pl. n-verziós programozás, belső állapotmentés)
- Idő-redundancia
  - Pl. többszöri végrehajtás, ellenőrző számítások
- A gyakorlatban a fenti elemek vegyesen jelennek meg.
  - Pl. redundáns kód alkalmazása hardvertöbblettel jár.
  - Pl. szoftveres belső ellenőrzés, állapotmentés időtöbblettel jár.

© BME-MIT 2026

Viselkedés hiba esetén

- Hiba észlelése esetén után el kell döntenie a rendszernek, hogy hogyan viselkedik

- Fail-stop-system
  - Hiba esetén a rendszer azonnal biztonságos állapotba megy. Ez az emergency stop viselkedést jelenti, tehát rögtön lekapcsolja magát.
- Fail-reduced-system
  - Hiba esetén a rendszer csökkentett módban folytatja a működését. Ez általában autókban az ún. limp-home mode, például a váltóvezérlő 4-esbe ragad.
- Fail-operational-system
  - Valamilyen backup segítségével a rendszer folytatja a viselkedését, amíg a tartalék üzemképes.



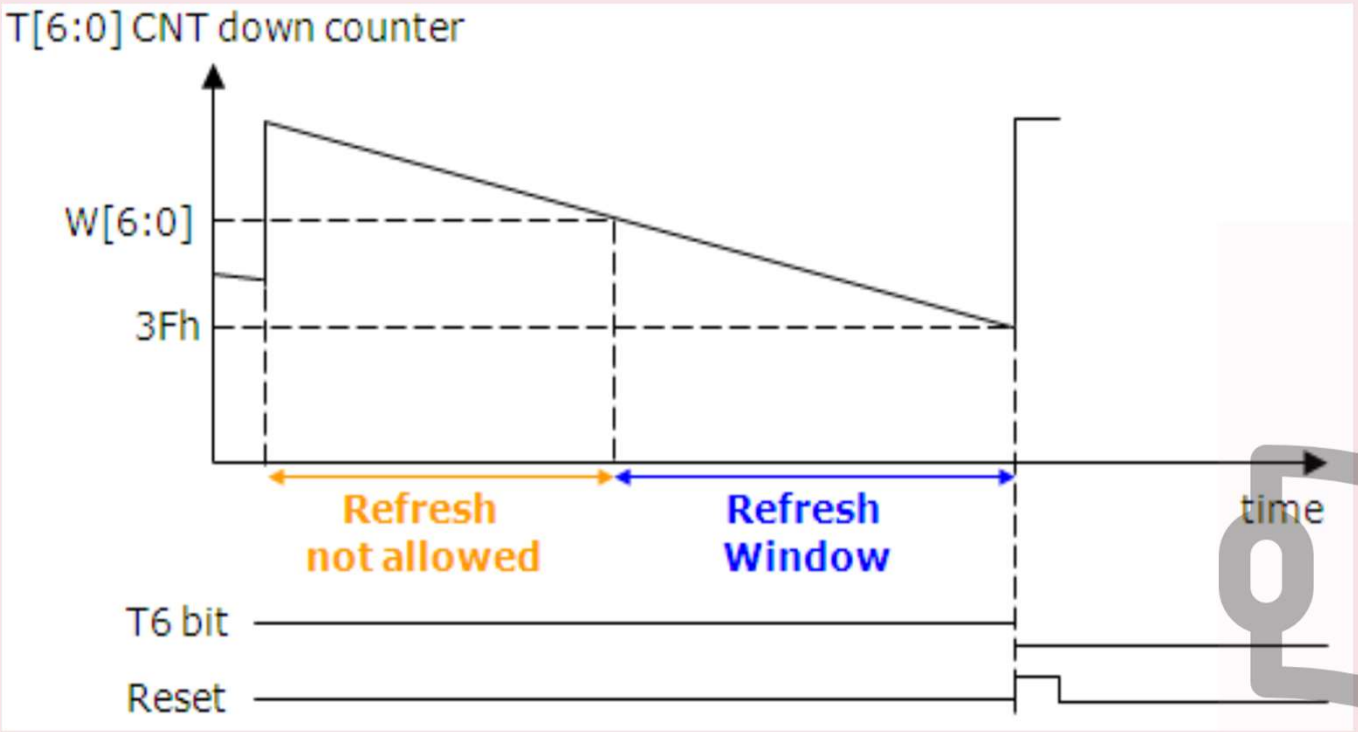
Tartalom

- Beágyazott szoftverek jellemző működési módjainak összefoglalása
- Bootloader
- Inicializációs mód
- Szervíz, kalibrációs mód
- Hibamód
  - Funkcionális biztonság áttekintés
  - Minimális elvárás a hibamóddal szemben

© BME-MIT 2026

Minimális szoftveres hibafedés

- Szoftveres hibák fedésére javasolt legalább egy windowed watchdog használata független órajelről
- A watchdog törlés mindig a főciklusban, vagy oprendszeres kód esetében a legalacsonyabb prioritású taszkban legyen benne



Minimális elvárás a hibamóddal szemben

- Implementáljunk egy hibatárolót
  - Azonosítsunk lehetséges hibaokokat, és ezeket üssük be a hiba tárolóba
  - Határozzuk meg a hiba tároló formátumát: Pl. idő bélyeg, hibakód, egyéb paraméterek x byte-on, és minden sor így legyen szervezve
  - Példa hiba beütésére: jelenleg nem várt megszakítás esetében egy végtelen ciklusba megyünk, de itt célszerű lenne inkább beütni egy hibát azzal a paraméterrel, hogy milyen IT volt ez, és utána reszetet dobni
- Tegyük lehetővé a hibatároló kiolvasását
  - Minimális megvalósítás esetén a konzolra való printf induláskor
- Biztosítsuk, hogy tartós hiba esetén biztonságos állapotba kerüljünk be
  - Jó dolog a watchdog, meg hiba esetén reszet, de ha n alkalom után is folyamatosan újra indulunk, akkor célszerű inkább biztonságosan leállni.



Példa egy modern safety mikrovezérlőre: TI Hercules sorozat

